

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – Industry Problem Solving Task

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO4 – Precision	Assessment:	Assessment Tool 2 – Industry Problem Solving Task
Weightage:	35% of CIA		
CO5 Statement:	Formulate context-free grammars, feature structures, probabilistic parsing techniques and to construct parsing frameworks. (BL-4)		
Student Name:	RENEE VINCENT ARS	Reg. No.:	192424161
Section / Batch:	BTECH AI & DS	Date:	14-08-2026

Code of Conduct

/ Renee Vincent ASR-192424161 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Renee Vincent

Instructions

- Read each industry scenario carefully before answering.
- Analyze the given problem from a semantic analysis perspective.
- Provide structured and logical answers with proper justification.
- Support your analysis using examples from the given scenario wherever applicable.
- Use suitable diagrams, semantic representations, logical predicates, workflows, architectures, or tables whenever necessary.
- Clearly state assumptions if additional information is required.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Representing Meaning & Meaning Structure of Language	Analyze semantic interpretation of user queries, identify ambiguity in meaning representation, evaluate semantic processing limitations, and improve understanding in banking chatbot applications.	Q1
Syntax-Driven Semantic Analysis	Analyze multiple interpretations of spoken commands, evaluate parsing-related ambiguity, and assess semantic extraction in real-time voice assistant systems.	Q2
First-Order Predicate Calculus, Representing Linguistically Relevant Concepts, Syntax-Driven Semantic Analysis	Design a semantic processing architecture for healthcare reports, model relationships among entities and actions, represent linguistic concepts, and improve semantic extraction accuracy.	Q3

Annexure A – Industry Problem Solving Task

1. A company is developing an AI-powered customer support chatbot for banking services. The chatbot uses Context-Free Grammar (CFG) for parsing user queries. However, it faces issues such as:

Incorrect interpretation of ambiguous sentences
Failure to handle subject–verb agreement
Inefficient parsing for long queries

Example user query:

“Show me the transactions with the card from last month”

- Analyze the ambiguity present in the given sentence using CFG
- Evaluate the limitations of CFG in handling real-world conversational queries
- Evaluate and propose an improved solution using PCFG, feature structures, or Earley parsing
- Justify how your proposed method improves accuracy and efficiency in an industry setting

2. A voice assistant system (like Alexa/Siri-type applications) processes spoken commands using parsing techniques. The system currently uses top-down parsing, but faces:

Backtracking issues
Poor handling of incomplete or ambiguous input
Difficulty in real-time response

Example command:

“Book a flight to Delhi with a window seat”

- Analyze how the given sentence can lead to multiple parse structures
- Analyze the limitations of top-down parsing in this real-time application
- Analyze how Earley parsing can improve handling of ambiguity and partial inputs
- Compare and analyze the performance of top-down vs Earley parsing for such industry use cases

3. A healthcare company is developing an AI system to analyze patient medical reports and automatically extract critical information such as diagnosis, prescribed treatments, and follow-up actions.

The system must handle:
Complex and lengthy medical sentences
Ambiguity in sentence structure
Accurate subject-verb agreement
Relationships between medical actions and entities (sub-categorization)
Real-time processing for hospital use

Example input:

"The doctor who reviewed the patient last week recommends starting medication and scheduling a follow-up visit in Chennai."

The current system, based only on basic Context-Free Grammar (CFG), fails to correctly interpret relationships and produces inconsistent outputs.

- Design a complete NLP architecture integrating:
CFG for syntactic structure
Probabilistic CFG (PCFG) for ambiguity resolution
Feature Structures for agreement handling

Efficient parsing techniques

b) Develop a step-by-step workflow showing how the system processes the given medical sentence from input to structured output (diagnosis, action, location).

c) Create a method to:

Incorporate sub-categorization frames for medical

verbs Handle real-time data processing

Improve accuracy and scalability in a hospital environment

1. CFG-BASED BANKING CHATBOT

a) CFG Ambiguity Analysis

Given Query

"Show me the transactions with the card from last month."

The sentence can have multiple syntactic interpretations because the phrases **"with the card"** and **"from last month"** can attach to different parts of the sentence.

Interpretation 1: Card as Transaction Method

Show [the transactions [with the card] [from last month]].

Meaning:

The user wants to see transactions that were made **using the card** during **last month**.

Structure:

$S \rightarrow VP$

$VP \rightarrow \text{Verb} + NP$

$NP \rightarrow \text{Det} + N + PP + PP$

Here:

- "transactions" $\rightarrow$ Noun
- "with the card" $\rightarrow$ Prepositional Phrase
- "from last month" $\rightarrow$ Prepositional Phrase

This is the most likely interpretation in a banking context.

Interpretation 2: Card Associated with Transactions

The phrase **"with the card"** can also be interpreted as modifying the noun "transactions", while **"from last month"** may modify either the transactions or the card.

This creates structural ambiguity.

A CFG can generate both possible structures because it primarily considers grammatical rules and does not inherently understand the user's banking intention.

Main Ambiguity

The phrase:

"from last month"

could potentially modify:

transactions

or another noun phrase within the sentence.

Therefore, the CFG may generate multiple parse trees for the same query.

b) Limitations of CFG

A basic CFG represents language using grammatical production rules such as:

$S \rightarrow NP VP$

NP → Det N

NP → NP PP

VP → V NP

Although CFG is useful for basic sentence parsing, it has several limitations in real-world banking chatbots.

1. Structural Ambiguity

CFG can produce multiple valid parse trees without knowing which interpretation the customer actually intended.

2. Lack of Context

CFG does not naturally understand banking concepts such as:

- Transactions
- Credit cards
- Account statements
- Payment history
- Last month

It focuses mainly on grammatical structure.

3. Subject-Verb Agreement

A basic CFG may accept grammatically incorrect combinations unless separate rules are created.

For example:

"Customer want transactions."

A simple CFG may parse this unless agreement constraints are explicitly included.

4. Long Queries

Long conversational queries can generate a very large number of possible parse trees.

This increases:

- Processing time.
- Memory requirements.
- Parsing complexity.

5. Limited Conversational Understanding

Users may type informal queries such as:

"Show my card spends last month."

or:

"What did I spend last month?"

A strict CFG may fail because users do not always follow predefined grammatical structures.

6. No Probability Ranking

A basic CFG cannot determine which of several valid parses is more likely.

For a banking chatbot, the system needs to select the interpretation that best matches the user's intent.

c) Improved Solution

A suitable solution is to combine **PCFG + Feature Structures + Efficient Parsing**.

1. PCFG

A Probabilistic Context-Free Grammar assigns probabilities to grammar rules.

For example:

NP → NP PP [0.40]

NP → Det N [0.60]

When multiple parse trees are possible, the system can calculate their probabilities and select the most likely interpretation.

For the query:

"Show me the transactions with the card from last month."

the PCFG can assign a higher probability to the interpretation:

transactions + with the card + from last month

because this structure is common in banking queries.

2. Feature Structures

Feature structures can represent grammatical properties such as:

Number = Singular/Plural

Person = First/Second/Third

Tense = Present/Past

For example:

Customer → Number = Singular

wants → Number = Singular

This helps enforce subject-verb agreement.

3. Earley Parsing

Earley parsing is suitable for natural-language queries because it can efficiently handle general CFGs and ambiguous structures.

It is especially useful when:

- The grammar is complex.

- The sentence is long.
- Multiple interpretations are possible.
- The input is conversational.

Proposed Architecture

User Query



Tokenization



Feature Extraction



PCFG/Earley Parser



Candidate Parse Trees



Probability Ranking



Semantic Interpretation



Banking Intent



Chatbot Response

d) Industry Benefits

The proposed approach improves the banking chatbot in several ways.

1. Better Ambiguity Resolution

PCFG assigns probabilities to different parse structures and selects the most likely interpretation.

For example:

"Show me the transactions with the card from last month."

can be interpreted as a request for **last month's card transactions**.

2. Improved Grammatical Accuracy

Feature structures enforce grammatical properties such as subject-verb agreement.

This reduces incorrect interpretations of user queries.

3. Efficient Long-Query Processing

Earley parsing can process complex and ambiguous CFG structures without requiring a separate grammar for every possible sentence pattern.

4. Better Intent Detection

The parser can extract important banking entities:

- **Action:** Show
- **Object:** Transactions
- **Payment Method:** Card
- **Time Period:** Last month

The resulting semantic representation can be:

QUERY(Transaction, Card, LastMonth)

This representation can then be passed to the banking system to retrieve the appropriate transaction records.

5. Improved User Experience

Users can express requests naturally instead of following fixed sentence patterns.

Examples:

"Show my card transactions from last month."

"What did I spend using my card last month?"

"Display last month's card transactions."

The system can map all these queries to the same underlying intent.

6. Industry-Level Reliability

For a banking chatbot, accurate parsing is important because incorrect interpretation could expose the wrong transaction information or perform an incorrect banking operation.

A combination of **PCFG**, **feature structures**, and **Earley parsing** provides better ambiguity handling, grammatical validation, and efficient parsing than a basic CFG alone.

Final Conclusion

A basic CFG can identify the grammatical structure of **"Show me the transactions with the card from last month"**, but it may produce multiple valid interpretations and cannot reliably determine the user's intended meaning. **PCFG** improves ambiguity resolution through probability, **feature structures** handle grammatical constraints such as subject-verb agreement, and **Earley parsing** efficiently handles complex and ambiguous queries. Together, these techniques can provide more accurate semantic interpretation and more reliable intent detection for an industry-scale banking chatbot.

2. VOICE ASSISTANT PARSING SYSTEM

a) Multiple Parse Structures

Given Command

"Book a flight to Delhi with a window seat."

The sentence contains an ambiguity in the phrase **"with a window seat."** The phrase can be attached to different parts of the sentence.

Parse Structure 1: Window Seat as Flight Requirement
Book [a flight [to Delhi] [with a window seat]].

Meaning:

The user wants to **book a flight to Delhi** and additionally requests **a window seat**.

Structure:

$S \rightarrow VP$

$VP \rightarrow V NP$

$NP \rightarrow Det N PP PP$

Here:

- "Book" → Verb
- "a flight" → Object
- "to Delhi" → Destination
- "with a window seat" → Seat preference

This is the most likely interpretation for a flight-booking voice assistant.

Parse Structure 2: "With a Window Seat" Attached Differently

The parser may alternatively associate the phrase **"with a window seat"** more closely with the flight

noun phrase:

[a flight [to Delhi] [with a window seat]]

Although the intended meaning remains similar, the syntactic attachment differs.

Main Ambiguity

The prepositional phrase:

"with a window seat"

can have different attachment possibilities in a general grammar.

A parser therefore needs contextual and probabilistic information to select the interpretation that best represents the user's intention.

b) Limitations of Top-Down Parsing

Top-down parsing begins with the start symbol and attempts to generate the input sentence by expanding grammar rules.

For example:

$S \rightarrow VP$

↓

$VP \rightarrow V NP$

↓

$NP \rightarrow Det N PP$

The parser continues expanding rules until it matches the input.

1. Backtracking

When an incorrect grammar rule is selected, the parser must return to an earlier point and try another rule.

For a voice assistant, repeated backtracking can increase processing time.

2. Ambiguous Sentences

Top-down parsing can generate multiple possible structures for phrases such as:

"with a window seat"

The system may need to explore several alternatives before finding the appropriate interpretation.

3. Incomplete Input

Speech recognition can produce incomplete commands such as:

"Book a flight to..."

The user may pause before saying the destination.

A traditional top-down parser may struggle because it expects a complete grammatical structure.

4. Real-Time Constraints

Voice assistants must respond quickly. Excessive backtracking increases computational overhead and can cause delays.

5. Error Propagation

If an early grammar decision is incorrect, subsequent parsing may also fail, requiring additional backtracking.

6. Limited Flexibility

Spoken language often contains:

- Pauses.
- Corrections.
- Repeated words.
- Incomplete phrases.
- Informal structures.

A basic top-down parser is not naturally designed to handle these characteristics efficiently.

c) Earley Parsing for Ambiguity and Partial Input

Earley parsing is a dynamic-programming parsing technique that can efficiently handle general context-free grammars.

It uses three main operations:

1. Prediction

Predicts possible grammar rules based on the current state.

2. Scanning

Matches an expected grammar symbol with the next input word.

3. Completion

Completes a grammar rule when all of its components have been recognized.

Handling Ambiguity

For:

"Book a flight to Delhi with a window seat."

Earley parsing can maintain multiple possible interpretations simultaneously rather than repeatedly restarting the parsing process.

Possible structures can include:

Flight + Destination + Seat Preference

The parser can maintain the competing possibilities and select the appropriate parse using additional probabilities or semantic information.

Handling Partial Input

Suppose the user says:

"Book a flight to..."

The Earley parser can maintain a partial parse state indicating that a destination is expected.

When the user continues:

"Delhi..."

the parser can continue from the existing state rather than restarting the entire parsing process.

Advantages for Voice Assistants

- Handles ambiguous grammar.
- Supports incomplete input.
- Avoids unnecessary repeated parsing.
- Maintains multiple possible parse states.
- Works with general CFG grammars.
- Can be extended with probabilities.
- Suitable for incremental speech processing.

d) Top-Down vs Earley Parsing

Feature	Top-Down Parsing	Earley Parsing
Parsing direction	Start symbol to input	Dynamic chart-based parsing
Ambiguity	Difficult to manage efficiently	Handles multiple parses
Backtracking	Often required	Reduced through chart states
Partial input	Poor handling	Better handling
Long sentences	Can become expensive	More systematic
General CFG	Limited depending on implementation	Supports general CFG
Real-time use	Can suffer from delays	Better suited to complex input
Memory usage	Usually lower for simple cases	Higher due to chart storage
Implementation	Relatively simple	More complex
Error recovery	Limited	Better support for partial structures

Performance Analysis

For a simple command such as:

"Book a flight."

top-down parsing can be fast because there is little ambiguity and only a few grammar rules need to be explored.

However, for a longer command:

"Book a flight to Delhi with a window seat."

there are more possible structures. Top-down parsing may need additional backtracking to determine the attachment of **"with a window seat."**

Earley parsing stores partial parsing results in a chart. If multiple interpretations exist, these states can be maintained without repeatedly starting the parsing process from the beginning.

Real-Time Industry Recommendation

For a voice assistant processing simple, predictable commands, **top-down parsing can be efficient and easy to implement.**

For a large-scale voice assistant handling:

- Ambiguous commands.
- Long sentences.
- Incomplete speech.
- Natural conversational language.
- Real-time user corrections.

Earley parsing is more suitable.

A practical system can further combine Earley parsing with **probabilistic scoring and semantic analysis** to select the most likely interpretation.

Final Conclusion

Top-down parsing is simple and effective for short, well-structured commands, but backtracking and ambiguity can reduce its performance in real-time voice applications. Earley parsing maintains multiple partial parsing states and can handle ambiguity and incomplete input more effectively. Therefore, for an industry-scale voice assistant, an **Earley-based parser combined with probabilistic and semantic information** provides greater flexibility and reliability for complex spoken commands.

3. NLP ARCHITECTURE FOR HEALTHCARE MEDICAL REPORT ANALYSIS

a) Complete NLP Architecture

Given Sentence

"The doctor who reviewed the patient last week recommends starting medication and scheduling a follow-up visit in Chennai."

The system must identify the grammatical structure, resolve ambiguity, maintain agreement, identify medical actions and entities, and produce structured information.

Proposed Architecture

Medical Report Input

↓

Text Preprocessing

↓

Tokenization

↓

Medical Named Entity Recognition

↓

CFG Parsing

↓

PCFG Ambiguity Resolution

↓

Feature Structure Analysis

↓

Sub-Categorization Analysis

↓

Semantic Role Labeling

↓

Efficient Parsing / Validation



Structured Medical Output

1. Text Preprocessing

The input medical report is first cleaned and divided into sentences and tokens.

Example tokens:

"The / doctor / who / reviewed / the / patient / last / week / recommends / starting / medication / and / scheduling / a / follow-up / visit / in / Chennai."

This stage also normalizes abbreviations, punctuation, and common medical terminology.

2. CFG for Syntactic Structure

CFG rules describe the grammatical structure.

Example:

$S \rightarrow NP VP$

$NP \rightarrow Det N$

$NP \rightarrow NP RelClause$

$VP \rightarrow V NP$

$VP \rightarrow V GerundPhrase$

These rules help identify:

- Subject
- Verb
- Object
- Relative clause
- Location
- Actions

The main structure can be represented as:

Subject: The doctor who reviewed the patient last week

Main Verb: recommends

Actions: starting medication, scheduling a follow-up visit

Location: Chennai

3. PCFG for Ambiguity Resolution

A normal CFG may produce multiple valid parse trees.

PCFG assigns probabilities to grammar rules and selects the most probable interpretation.

For example:

recommends [starting medication and scheduling a follow-up visit]

is more likely than treating "starting" and "scheduling" as unrelated sentence structures.

This helps resolve ambiguity in long medical sentences.

4. Feature Structures

Feature structures store grammatical properties such as:

Person = Third
Number = Singular
Tense = Present

For:

doctor recommends

Doctor = singular

Recommends = third-person singular

Therefore, the system validates:

doctor → recommends

rather than incorrectly generating:

doctor → recommend

5. Efficient Parsing

For long and complex medical reports, the system can use:

- Earley parsing.
- Chart parsing.
- Dynamic programming.
- Incremental parsing.

These techniques avoid repeatedly parsing the same sentence structures and improve processing efficiency.

6. Medical Semantic Analysis

The system identifies medical entities and relationships.

For example:

Doctor → Recommends → Medication

Doctor → Recommends → Follow-up Visit

Follow-up Visit → Location → Chennai

b) Step-by-Step Processing Workflow

Step 1: Input

The system receives:

"The doctor who reviewed the patient last week recommends starting medication and scheduling a follow-up visit in Chennai."

Step 2: Tokenization

The sentence is divided into individual tokens.

The | doctor | who | reviewed | the | patient | last | week | recommends | starting | medication | and | scheduling | a | follow-up | visit | in | Chennai

Step 3: Entity Recognition

Important entities are identified:

Doctor → **Medical Professional**

Patient → **Person**

Medication → **Treatment**

Follow-up Visit → **Medical Procedure/Action**

Chennai → **Location**

Step 4: CFG Parsing

The system identifies the main grammatical relationships.

Main Subject: Doctor

Main Verb: recommends

Object/Complement: starting medication and scheduling a follow-up visit

The phrase:

"who reviewed the patient last week"

is recognized as a relative clause modifying **doctor**.

Step 5: PCFG Resolution

The PCFG evaluates possible structures and selects the most probable interpretation:

Doctor recommends [starting medication] and [scheduling a follow-up visit in Chennai].

This avoids incorrectly associating "last week" with the recommendation action.

Step 6: Feature Validation

The system checks agreement:

Doctor = **singular**

recommends = **third-person singular**

Therefore:

Doctor recommends → **Correct**

Step 7: Sub-Categorization

The verb **"recommends"** is associated with actions or propositions.

Its simplified frame can be:

recommend(Agent, Action)

Therefore:

recommend(Doctor, starting medication)

and:

recommend(Doctor, scheduling follow-up visit)

Step 8: Semantic Role Analysis

The system generates:

Doctor → **Agent**
Patient → **Patient/Entity**
Medication → **Treatment**
Follow-up Visit → **Medical Action**
Chennai → **Location**

Step 9: Structured Output

A possible structured representation is:

Diagnosis: Not explicitly stated

Agent: Doctor

Patient: Patient

Action 1: Start Medication

Action 2: Schedule Follow-up Visit

Location: Chennai

Time Reference: Last Week

Semantic Representation

```
RECOMMEND(  
  Doctor,  
  START(Medication),  
  SCHEDULE(FollowUpVisit, Chennai)  
)
```

Final Structured Table

Information	Extracted Value
Diagnosis	Not explicitly stated
Agent	Doctor
Patient	Patient
Action 1	Start Medication
Action 2	Schedule Follow-up Visit
Location	Chennai
Time Reference	Last Week

c) Sub-Categorization and Real-Time Processing

1. Medical Sub-Categorization Frames

Sub-categorization describes what types of arguments a verb can take. Medical verbs can be associated with predefined frames.

Examples:

prescribe

prescribe(Doctor, Medication, Patient)

recommend

recommend(Doctor, Treatment/Action)

diagnose

diagnose(Doctor, Disease, Patient)

schedule

schedule(Staff, Appointment, Time/Location)

administer

administer(Doctor/Nurse, Medication, Patient)

These frames help the system determine which entities can logically participate in a medical action.

2. Applying the Frame

For:

"Doctor recommends starting medication."

The system identifies:

Agent = Doctor

Action = Starting

Object = Medication

Therefore:

RECOMMEND(Doctor, START(Medication))

For:

"scheduling a follow-up visit in Chennai"

the system identifies:

Action = Schedule

Object = Follow-up Visit

Location = Chennai

Therefore:

SCHEDULE(FollowUpVisit, Chennai)

Real-Time Processing Method**Stage 1: Streaming Input**

Medical reports can be processed sentence-by-sentence rather than waiting for the entire document.

Stage 2: Incremental Parsing

Use incremental or chart-based parsing to update the interpretation as new words arrive.

For example:

"The doctor recommends..."

The system creates a partial interpretation.
When it receives:
"starting medication..."
the semantic representation is updated.

Stage 3: Parallel Processing

Different NLP components can process information simultaneously:

- NER → Medical entities.
- Parser → Grammar.
- PCFG → Ambiguity.
- Semantic analyzer → Relationships.
- Sub-categorization → Medical actions.

This reduces processing time.

Stage 4: Medical Knowledge Base

A medical terminology database or ontology can validate extracted concepts and relationships.
For example:

Medication → Treatment

Chennai → Geographic Location

Follow-up Visit → Clinical Action

Stage 5: Confidence Scoring

Each extracted relationship can receive a confidence score.

For example:

Doctor → recommends → Medication 0.96

Doctor → recommends → Follow-up Visit 0.94

Follow-up Visit → Chennai 0.91

Low-confidence results can be flagged for human review.

Improving Accuracy and Scalability

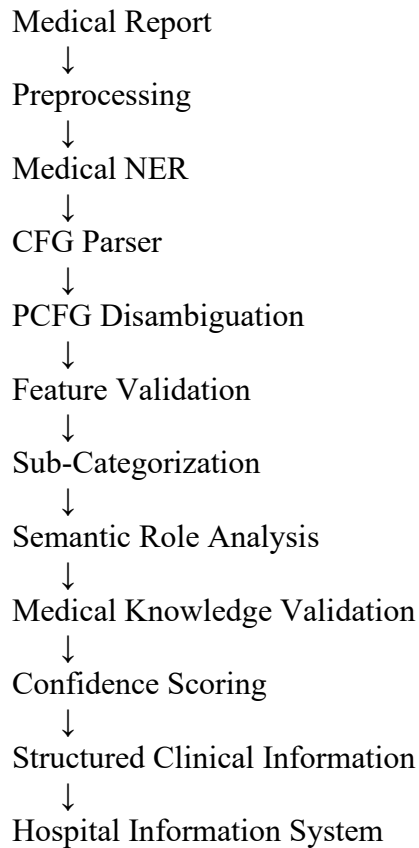
Accuracy Improvements

- Use medical-specific PCFG rules.
- Add medical sub-categorization frames.
- Use feature structures for agreement.
- Apply medical Named Entity Recognition.
- Use dependency parsing and semantic role labeling.
- Include negation detection.
- Include temporal relationship extraction.
- Use confidence-based validation.

Scalability Improvements

- Use chart/Earley parsing for complex structures.
- Process documents incrementally.
- Use parallel NLP pipelines.
- Cache frequently used grammar structures.
- Use optimized medical dictionaries.
- Separate preprocessing, parsing, and semantic services.
- Use batch processing for large historical records.

Hospital Environment Architecture



Final Conclusion

A basic CFG alone is insufficient for complex medical reports because it cannot reliably resolve ambiguity, enforce detailed grammatical features, or understand medical relationships. A combined architecture using **CFG for syntactic structure, PCFG for ambiguity resolution, feature structures for agreement, efficient Earley/chart parsing, medical NER, semantic role labeling, and sub-categorization frames** provides a more reliable solution. For the given sentence, the system correctly identifies the **doctor as the agent, medication as a treatment, follow-up visit as an action, Chennai as the location, and "last week" as a time reference**. This architecture can provide accurate, scalable, and near-real-time information extraction for hospital applications.

Rubrics for Calculation

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding ; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Q. No. / Criteria Level	Understanding of Industry Problem	Application of Engineering Concepts	Solution Design	Use of Tools	Analysis	Professionalism	Sub. Total	Total %
1								
2								
3								

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____